

BUILD



TEST



SELL

Networking in Series 60 Platform 2.0

September 24, 2003

Symbian Technology Day

Forum Nokia Hub, Helsinki

Marko Okkonen

Technology Expert, Forum Nokia

NOKIA

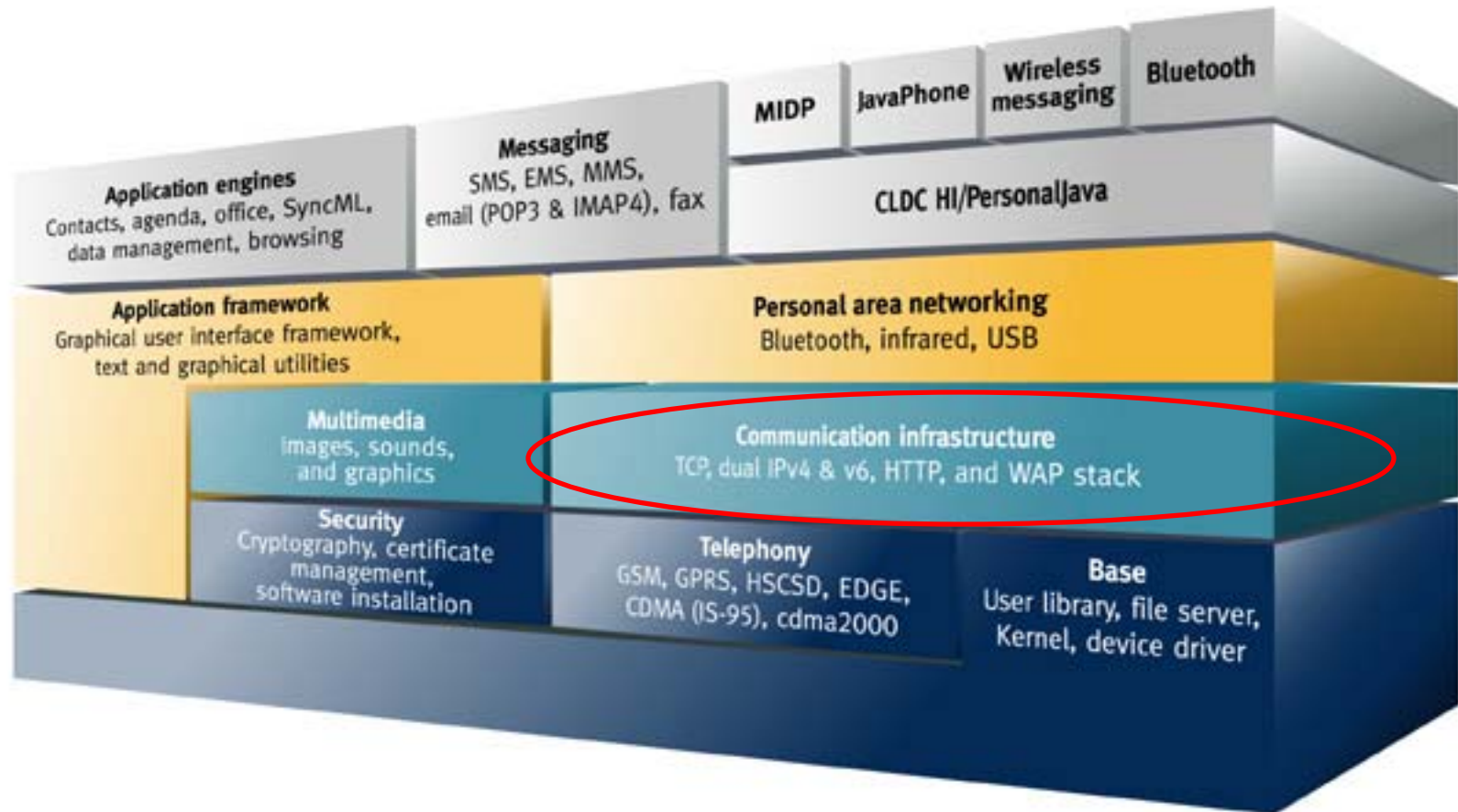
Outline

- Introduction to Networking
- Connection Management API
- HTTP Client API
- Communications Database
- Miscellaneous
- Summary

Prerequisites

- Symbian Essentials, or equivalent experience
- Some knowledge of Symbian networking

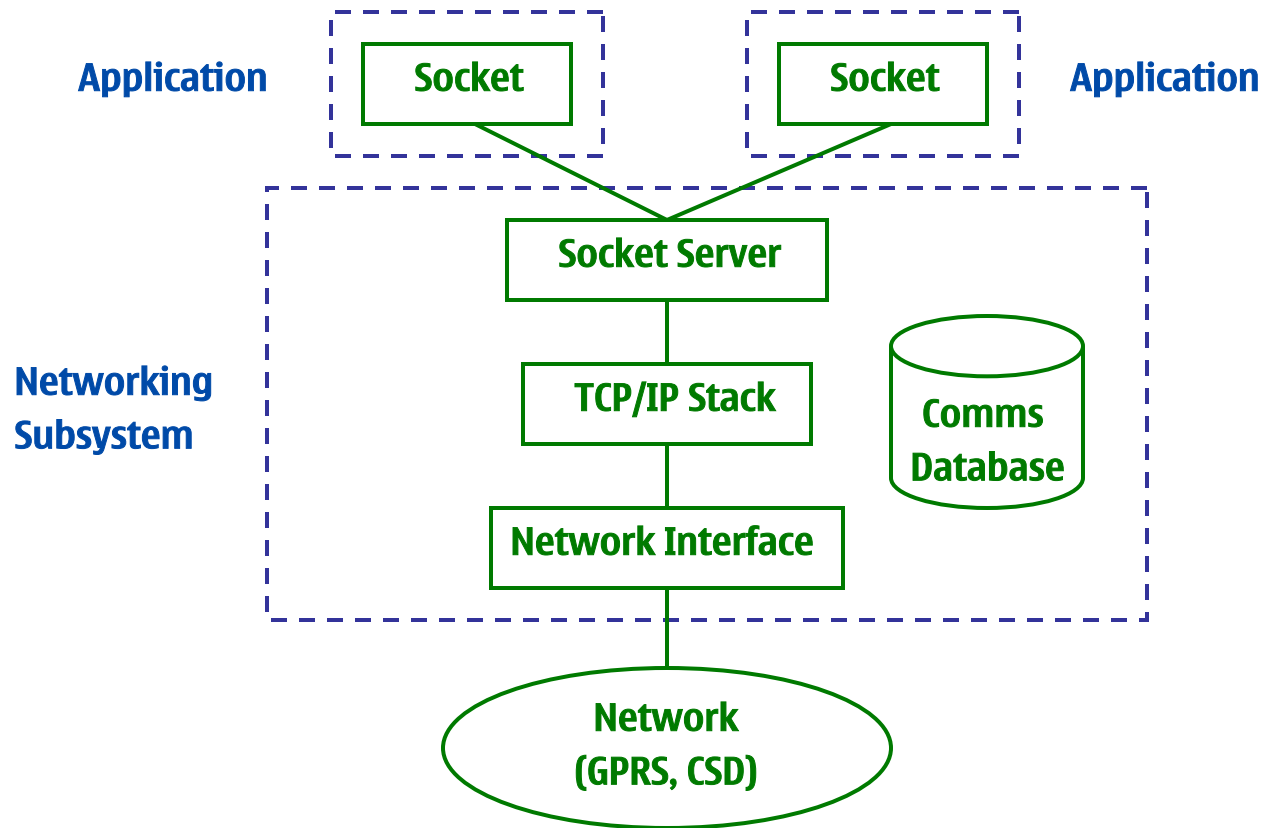
Introduction to Networking



Networking Overview

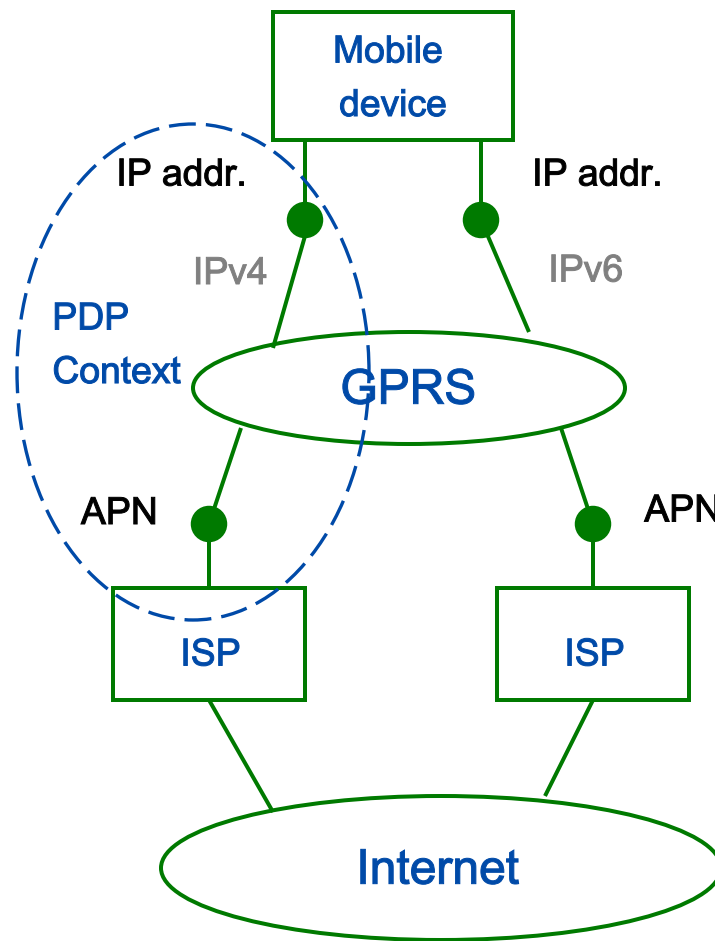
- What's new in the Series 60 v2.0?
 - IPv4/IPv6 dual stack
 - HTTP/1.1 client stack
 - Multihoming (i.e. multiple PDP contexts)
 - Connection Management API
 - Connection Manager application
- Bearers supported
 - GSM Circuit Switched Data
 - GSM High-Speed Circuit Switched Data
 - GPRS
 - Readiness for 3G (WCDMA, CDMA2000)

Networking Architecture

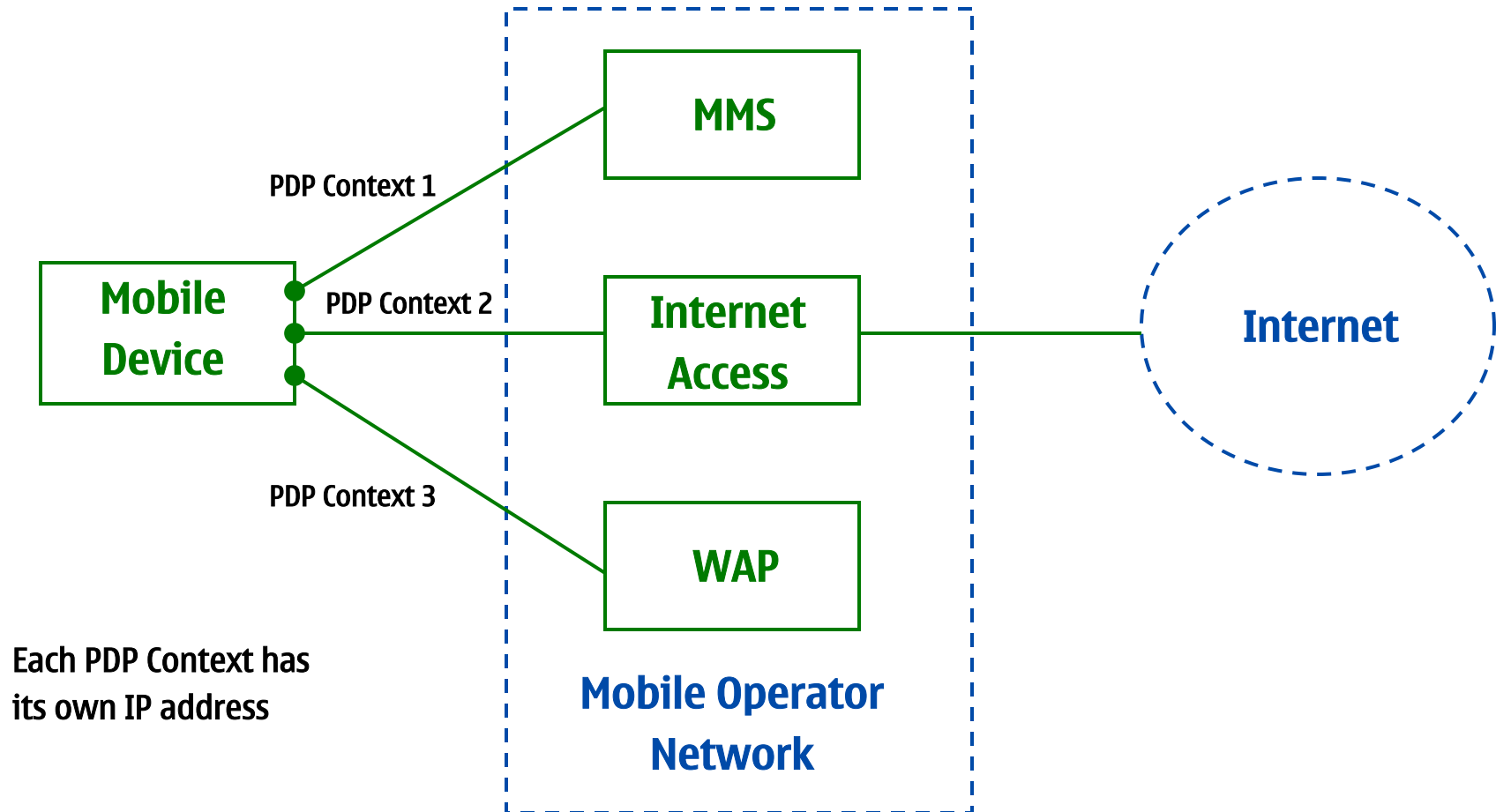


Mobile Internet over GPRS

- “Always on”
- PDP context
 - PDN type (e.g. IPv4)
 - PDN address (e.g. IP address)
 - QoS
 - APN
- Single-/multihoming
- Usually dynamic IP address, because of Network Address Translation Gateway (NAT), which implementation varies between the operators



Multihoming



Connection Management API

Connection Management Overview

- RConnection class implements the Connection Management API
- Provides an interface for creating and managing connections
- Supports multi-homing, which allows multiple circuit/packet switched data connections to be active
- RConnection replaces old APIs: RNif, RGenericAgent, CIntConnectionInitiator and CInternetConnectionManager
- Wrappers for old API exists for the binary compatibility, but multi-homing is not supported

Functionalities of the RConnection

- Opening and closing the connection
- Starting a connection, which means associating it with a new underlying interface
- Attaching the RConnection instance to an existing interface
- Obtaining progress information and notification during connection start-up
- Notifying when subconnections come up and go down
- Reading CommDB fields specific to an active connection
- Collecting statistical information on the network connection and subconnections

Opening Socket connection

- Two versions of RSocket::Open() method
- One for implicit connection startup (old)

```
TInt Open(RSocketServ& aServer, TInt addrFamily, TInt  
sockType, TInt protocol);
```

- Another for explicit connection startup (new)

```
TInt Open(RSocketServ& aServer, TInt addrFamily, TInt  
sockType, TInt protocol, RConnection& aConnection);
```

Implicit Connection Startup

```
RSocketServ ss;  
RSocket sock;  
TRequestStatus status;  
  
// Connect to the socket server  
ss.Connect();  
  
// Open a TCP socket  
sock.Open(ss, KAfInet, KSockStream, KProtocolInetTcp);  
  
_LIT(KRasAddr, "10.159.24.13");  
const TInt KEchoPort = 7;  
  
TInetAddr destAddr;  
destAddr.Input(KRasAddr);  
destAddr.SetPort(KEchoPort);  
  
// Request the Socket to connect to the destination (implicit Connection)  
sock.Connect(destAddr, status);
```

Explicit Connection Setup

```
RSocketServ ss;
RConnection conn;
TRequestStatus status;
_LIT(KRasAddr, "10.159.24.13");
const TInt KEchoPort = 7;

TInetAddr destAddr;
destAddr.Input(KRasAddr);
destAddr.SetPort(KEchoPort);

// Connect to the socket server
ss.Connect();

// Open an RConnection object.

conn.Open(ss);

// Create overrides
TCommDbConnPref prefs
prefs.SetDialogPreference(ECommDbDialogPrefDoNo
    tPrompt);
prefs.SetDirection(ECommDbConnectionDirectionOu
    tgoing);
prefs.SetIapId(4);

// Start an Outgoing Connection with overrides
conn.Start(prefs);
```

```
// Open a Socket associated with the connection
RSocket sock;
sock.Open(ss, KAfInet, KSockStream,
    KProtocolInetTcp, conn);

// Request the Socket to connect to the
// destination
sock.Connect(destAddr, status);
```

Attaching to a Connection

```
// Open a session with the socket server
RSocketServ socketServer;
TInt err(KErrNone);
err = socketServer.Connect();
If (err != KErrNone) return err;

//Open a subsession with the socket server for the Rconnection
RConnection myConnection;
err = myConnection.Open(socketServer);
If (err != KErrNone) return err;

//Start or attach the RConnection to an interface on the server
TUint connectionCount;
//Enumerate currently active connections across all socket servers
err = myConnection.EnumerateConnections(connectionCount);
If (err != KErrNone) return err;

If (connectionCount == 0) // if no connections are started
{
    err = myConnection.Start();
}
else
{
    TPckgBuf<TConnectionInfoV2> connectionInfo;
    err = myConnection.GetConnectionInfo(1, connectionInfo); // 1 = first active connection
    if (err!=KErrNone) return err;
    err = myConnection.Attach(connectionInfo, EAttachTypeMonitor);
}
```

Disabling Inactivity Timer

- Connection inactivity timer can be disabled using `RConnection::SetOpt()` function instead of `RNif::DisableTimers()`, which was used before.

```
SetOpt(KCOLProvider, KConnDisableTimers, ETrue);
```

HTTP Client API

HTTP Features

- HTTP 1.0 and 1.1 Client stack with support for persistent connections, pipelining, and chunked transfer encoding
- Filters provided to support HTTP Redirection and HTTP Basic and Digest Authentication.
- Secure HTTP Connection supported with SSL 3.0 and TLS 1.0
- HTTP state management (i.e. cookies) is supported

HTTP Client API

- There are five key concepts used in the API: sessions, transactions, headers, data suppliers and filters.

Sessions

- A session encapsulates the client's HTTP activity over the duration of the client's execution.
- Usually one session at time, but can be several
- Each session has a set of properties, which define HTTP protocol, encoding and transport that applies for transactions carried out within the life of the session
- The session has also an associated set of filters
- Session class is provided by RHTTPSession

```
// Open a session using the default protocol HTTP/TCP  
void RHTTPSession::OpenL()
```

Transactions

- A transaction represents an interaction between the HTTP client and server - a client request and a server response
- Transactions are executed asynchronously within the client's process. The client is notified when events are available for each outstanding transaction
- Both the request and response portion of a transaction are composed of a header and an optional body
- The request portion of the transaction also specifies an HTTP Method together with a URI that specifies the resource held at the server
- The response portion of a transaction contains an HTTP status code and message

Transactions - continued

- Transaction class is provided by RHTTPTransaction and callback interface by MHTTPTransactionCallback

```
// Open a transaction
void RHTTPSession::OpenTransactionL(const TUriC8 &aURI,
    MHTTPTransactionCallback &aCallback,
    RStringF aMethod=RStringF())

// Callback function
virtual Tint RHTTPTransactionCallback::MHFRunL(
    RHTTPTransaction aTransaction,
    const THTTPEvent& aEvent) = 0
```

Transactions – continued 2

```
// Get the response
RHTTPResponse RHTTPTransaction::Response() const

// Get the request
RHTTPResquest RHTTPTransaction::Request() const

// Get the property set
RHTTPTransactionPropertySet
    RHTTPTransaction::PropertySet() const

// Submit the transaction
void RHTTPTransaction::SubmitL(
    THTTPFilterHandle aStart=THTTPFilter::EClient)
```

Headers

- The header may have zero or more fields
- Headers are encoded according the HTTP protocol and encoding that are used
- The headers class is provided by RHTTPHeaders

```
// Get request headers
RHTTPHeaders RHTTPRequest::GetHeaderCollection()
// Set a named field
RHTTPHeaders::SetFieldL(RStringF aFieldName,
                        THTTPHdrVal& aHeaderValue)

// Get response headers
RHTTPHeaders RHTTPResponse::GetHeaderCollection()
// Obtain a named header field's value
TInt RHTTPHeaders::GetField(RStringF aFieldName,
                            TInt aPartIdx, THTTPHdrVal& aHeaderValue) const
```

Data Suppliers

- The body portion of requests and responses is represented in the API as mix-in interface, which allows implementation of the classes that generate body data to be hidden
- API enables signalling between the client and transport in use, to that body data is consumed or emitted at a rate the client can support
- Clients can assemble the body piece by piece
- Data supplier classes must implement `MHTTPDataSupplier`

Data Suppliers - Continued

```
// Inform HTTP that new data is available  
RHTTPTransaction::NotifyNewRequestBodyPartL()  
  
// Obtain a data part from the supplier  
TBool MHTTPDataSupplier::GetNextDataPart(TPtrC8& aDataPart)  
= 0  
  
// Data will survive until ReleaseData() is called  
void MHTTPDataSupplier::ReleaseData() = 0
```

Filters

- Filters are add-on modules that provide additional behaviours to a session
- Behaviours may be triggered by
 - The presence of particular headers in a request or a response
 - By status codes in a response
 - Particular events, which occur on a transaction
E.g. client authentication, redirection or caching
- Filter classes must implement MHTTFilter

Using RConnection with RHTTPSession

```
// RSocketServ iSocketServ
// RConnection iConnection;

// Connection is already established, See Explicit Connection Setup slide

RHTTPSession httpSession;
httpSession.OpenL();

RHTTPConnectionInfo connInfo = httpSession.ConnectionInfo();
RStringPool pool = httpSession.StringPool();

connInfo.SetPropertyL(pool.StringF(HTTP::EHttpSocketServ, RHTTPSession::GetTable()),
    THTTPHdrVal(iSocketServ.Handle()));

connInfo.SetPropertyL(pool.StringF(HTTP::EHttpSocketConnection, RHTTPSession::GetTable()),
    THTTPHdrVal(iConnection.SubSessionHandle()));

httpSession.OpenTransactionL(uri, *this, pool.StringF(HTTP::EGET, RHTTPSession::GetTable()));
```

Communications Database

Comm Db Overview

- Comm Db stores information about ISPs, modem and GPRS settings, WAP, proxies, Bluetooth, connection preferences etc.
- Comm Db is based on Symbian DBMS architecture, hence it is a client/server architecture and provides safe access for multiple clients
- Client access to the database is provided by CCommsDatabase

New Tables

- Network
 - Contains ids for all available networks
- All Bearers
 - Timeout values
- Modem bearer
 - Replaces MODEM table
- LAN bearer
 - Contains agent, NIF and drivers needed to load to enable the bearer

Updated Tables

- IAP table
 - New columns: Network, Service Type (e.g. CSD, GPRS, LAN, WCDMA), Service, Weight, Bearer, Bearer Type
 - ISP_CHARGECARD moved to Dial Out Service table
- New Global fields
 - BEARER_AVAILABILITY_CHECK_TSY is to be used when checking to assess coverage of all possible bearers in order to provide a feasible list of IAPs
 - DEFAULT_NETWORK

Removed Tables

- Modem table has been replaced by Modem Bearer table
- Agent table has been removed and fields have been located to other tables

Example IAP Table

CommDb Name	CommDb Id	Network	Service Type	Service	Weight	Bearer	Bearer Type
Home RAS	1	1	CSD	1	1	1	MODEM
Radiolinja GPRS WAP	2	2	WCDMA Packet	1	1	2	MODEM
Radiolinja GPRS	3	3	WCDMA Packet	2	1	2	MODEM
Kolumbus CSD	4	4	CSD	2	1	3	MODEM
Home LAN	5	1	LAN	1	2	1	LAN

References

Network
Table

Service
Type
Tables

Specific
Service Type
Table

Specific
Bearer
Table

Bearer
Tables

API changes

- New function

```
OpenIAPTableViewMatchingNetworkLC(TUint32 aNetwork)
```

- Deprecared functions

- All agent-specific functions

```
GetCurrentDialOutSettingL(const TDesC& aSetting, TUint32& aValue)
```

```
GetCurrentDialInSettingL(const TDesC& aSetting, TUint32& aValue)
```

Miscellaneous

Application Behaviour

- User confirmation is required always before connection is made
- Application may have own settings for the default AP if selection dialog is not wanted to be shown
- AVOID HARD CODINGS WHERE POSSIBLE!

Summary

- New features include multihoming, IPv4/IPv6 dual stack
- Multihoming enables multiple simultaneous PDP contexts
- New APIs for Connection Management, which supports multihoming, and HTTP client stack
- Comms database has slightly changed -> new network id concept

Thank You!

See You in the next event!